DEVOPS

Change the Default SSH Port

Changing the SSH port from the default 22 is a basic security measure to protect against automated scanning and brute-force attacks.

- **Step 1: Edit the SSH configuration file.** Open the `sshd_config` file, located at `/etc/ssh/sshd_config`, with `sudo` and change the line `#Port 22` to `Port [your_new_port]`.
- **Step 2: Update your firewall rules.** You **must** allow traffic on the new port before you restart the SSH service. For `ufw`, use `sudo ufw allow [your_new_port]/tcp`.
- **Step 3: Restart the SSH service.** Apply the changes by restarting the service with `sudo systemctl restart sshd`.

Essential Connection Commands

- **Connect with the new port:** Use the `-p` flag to specify the port number when connecting: `ssh -p [your_new_port] user@host`.

Create a configuration file: To avoid typing the port number every time, you can create an SSH config file (`~/.ssh/config`) with the following entry:

```
Host my_server
  HostName your_server_ip
  Port your_new_port
  User your_username
```

- You can then simply connect by typing `ssh my_server`.

HOSTING LOCAL SERVERS

Step 1: Getting A Public IP Address from KTRN via Mango 4g
Step 2: Getting a working (Port forwarding enabled) router
Check if the public ip has common ports open 22,80,443
Step 3: Configuring ssh via port forwarding to listening port according to the response from the previous response.

Step 4: via ssh continue with server configuration for services to run, like gunicorn by creating a systemd service to ease the app starting on boot.

Hint:

```
[Unit]
Description=Gunicorn instance to serve our email sending App
After=network.target

[Service]
User=acr
Group=www-data
WorkingDirectory=/home/acr/services
ExecStart=/home/acr/services/venv/bin/gunicorn \
          --workers 2 \
          --bind 0.0.0.0:9000 \
          app:app

Restart=always
StandardOutput=append:/var/log/gunicorn/gunicorn.log
StandardError=append:/var/log/gunicorn/gunicorn-error.log

[Install]
WantedBy=multi-user.target
```

Step 5: Port forward 9000 to the local ip of the server with gunicorn listening

Step 6: Port forward **8433** for nginx in case 443 is blocked by **isp**

Local IP Address	Local Port Range	Protocol	Remote IP Address	Remote Port Range	Status	Comment	Select
192.168.0.253	5000	TCP+UDP	ANY	5000	Enabled	Flask_access	☐
192.168.0.253	443	TCP	ANY	443	Enabled	https	☐
192.168.0.253	80	TCP	ANY	80	Enabled	http	☐
192.168.0.253	22	TCP+UDP	ANY	2222	Enabled	SSH Access	☐
192.168.0.253	9	UDP	ANY	9	Enabled	Wake on WAN	☐
192.168.0.251	3322	TCP	ANY	3322	Enabled	SSH Access_	☐
192.168.0.251	8443	TCP	ANY	8443	Enabled	https	☐
192.168.0.251	9000	TCP	ANY	9000	Enabled	Gunicorn	☐

Delete Selected | Delete All | Reset

```
# HTTPS on custom port 8443
server {
    listen 8443 ssl;
    listen [::]:8443 ssl;
    server_name email.netpipo.com;

    ssl_certificate /etc/letsencrypt/live/email.netpipo.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/email.netpipo.com/privkey.pem;

    # Recommended SSL options
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers on;

    location / {
        proxy_pass http://105.178.106.24:9000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Step 7: Request **ssl certificates** manually by DNS 01 Challenge in order to not rely on port 80, 443 which are used defaultly by Certbot

Step 8: Enable ufw

NB: 1. Make Sure that nginx 8443 and gunicorn 9000 are portforwarding to the same local ip
2. If we want to host another server, we'll use different port listening to nginx and gunicorn as well

Remote MySQL Updates with Triggers

Scenario: You can't `UPDATE` a table remotely even though your user has full privileges.

Symptoms:

UPDATE tbl_imvoice SET balance = 5 WHERE id = 1;
ERROR 1143 (42000): SELECT command denied to user 'miseaurac'@'localhost' for column 'id'

Cause:

- Your table has a **trigger** (BEFORE or AFTER UPDATE).
- The trigger executes as the **DEFINER**, e.g., `miseaurac@localhost`.
- Even if your remote user (`eaurqb@<remote_ip>`) has ALL privileges, the **trigger definer lacks SELECT/INSERT privileges**, causing the UPDATE to fail.

Solution Options:

1. **Grant privileges to the trigger definer**

GRANT SELECT, UPDATE, INSERT ON mis_sandbox.tbl_imvoice TO 'miseaurac'@'localhost';
GRANT INSERT ON mis_sandbox.tbl_invoice_audit TO 'miseaurac'@'localhost';
FLUSH PRIVILEGES;

Extra Tips:

- Always check the **definer** of triggers:

SHOW TRIGGERS FROM mis_sandbox;

- Remote users need privileges **both on the table and any tables used by triggers**.
- Use `SHOW GRANTS FOR 'user'@'host';` to verify effective privileges.

- `CURRENT_USER()` shows the authenticated account MySQL is using — useful for debugging permission errors.

This pattern often explains mysterious `UPDATE` or `DELETE` errors when triggers are involved.